

LECTURE 2

State and Data Management: Hooks, Server Components, SWR

Course “Advanced Web Technologies” · Week 2 of 15 · 1 academic hour of lecture + independent study · Sections 6 and 8 are advanced and may be studied later

Lecture Objectives

After this lecture, you will be able to:

- explain what `useEffect` is actually for and why it is not a data-fetching mechanism;
- run independent data requests in a Server Component in parallel instead of creating a waterfall;
- place targeted `<Suspense>` boundaries so that a slow request does not block the entire page;
- understand when a Server Component alone is not enough and use SWR on the client intentionally;
- optionally, at your own pace: pass a seed from the server into the client cache and share state through `useReducer/useContext`.

1. From `useState` to the Other Hooks

In the previous lecture, we used one hook — `useState` — and one Client Component. Today’s lecture is denser, so let us set priorities immediately: the required minimum for this week is `useEffect` (Section 2) and SWR (Section 5), because they are needed for the laboratory work. Everything else — parallel loading, the seed pattern, `useReducer`, and `useContext` — is useful, but you can return to it at your own pace; these parts are marked separately in the text.

A short map for today: `useEffect` synchronizes React with something external; it is not a data store. SWR is a library for cases where a Server Component is no longer sufficient on its own. `useReducer` and `useContext` help organize state cleanly as it grows.

Running example for the lecture. Imagine a course catalog: the server must show the list on the first load, the user must be able to search courses without reloading the page, and a separate indicator must react to the browser state. These three tasks clearly show that “data” in React can have different origins and therefore requires different tools.

2. `useEffect`: for Synchronization, Not Fetching

The official React documentation even has an article titled “You Might Not Need an Effect.” This is not an exaggeration: before Server Components, a common way to obtain data in a component was `useEffect` with `fetch` inside, and many older tutorials still teach this pattern. In the App Router, this is unnecessary code in many cases.

✗ Unnecessary useEffect for data

```
"use client";
import { useState, useEffect } from "react";

function CourseStats() {
  const [stats, setStats] = useState(null);

  useEffect(() => {
    fetch("/api/stats")
      .then((r) => r.json())
      .then(setStats);
  }, []);

  if (!stats) return <p>Loading...</p>;
  return <p>Courses: {stats.total}</p>;
}
```

☑ The same thing with a Server Component

```
import { getStats } from "@lib/stats";

async function CourseStats() {
  const stats = await getStats();

  return <p>Courses: {stats.total}</p>;
}

// no 'use client', no useState,
// no loading state –
// it is not needed on the server
```

So when is `useEffect` actually needed? When you synchronize with something OUTSIDE React — a browser API, timer, subscription, or third-party library. A classic example is tracking network status:

Lifecycle walkthrough: the component appears → the Effect subscribes to an external system → the external system reports a change → React updates state → when the component is removed, cleanup unsubscribes. The presence of an external system is what distinguishes a useful Effect from an unnecessary one.

Quick check for students: if you remove `useEffect`, what stops being synchronized? If the answer is “nothing, the data simply will not load,” consider a Server Component first. If the answer is “we will stop listening to the browser, a timer, WebSocket, or a third-party widget,” the Effect is justified.

```
components/OnlineBadge.tsx
"use client";
import { useEffect, useState } from "react";

function OnlineBadge() {
  const [isOnline, setIsOnline] = useState(true);

  useEffect(() => {
    const onUp = () => setIsOnline(true);
    const onDown = () => setIsOnline(false);
    window.addEventListener("online", onUp);
    window.addEventListener("offline", onDown);

    return () => {
      window.removeEventListener("online", onUp);
      window.removeEventListener("offline", onDown);
    };
  }, []);

  return <span>{isOnline ? "online" : "offline"}</span>;
}
```

Rule to remember: if an Effect exists only to call fetch and put the result into state, that logic probably belongs in a Server Component rather than useEffect.

Practical counterexample: filtering an array, computing a full name from name + surname, or sorting an already loaded list does not require an Effect. These are calculations derived from existing state and are better performed directly during rendering. An Effect is not for “anything that happens after render”; it is for synchronization with the outside world.

3. Server Components in Depth: Parallel and Targeted Loading

Previously, we simply wrote await inside a component. If there are several awaits and they do not depend on one another, it is important not to turn them into a chain of waiting operations.

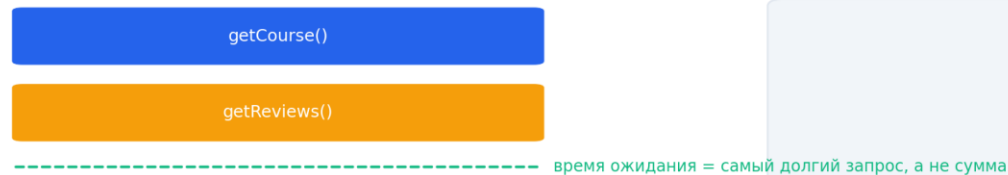
Waterfall против параллельной загрузки

Один и тот же Server Component, два независимых запроса данных

```
await getCourse(id); await getReviews(id);
```



```
const [course, reviews] = await Promise.all([...]);
```



Правило простое:

если два await не зависят друг от друга (второму не нужен результат первого) — запускайте оба СРАЗУ, потом await Promise.all. Если второй запрос использует id из первого — waterfall неизбежен, и это нормально.

Diagram A. Waterfall vs. parallel loading — the same result, different waiting time

The rule is simple: if the second request does not need the result of the first, start both immediately instead of awaiting them one after another, and wait for them with Promise.all. If the second request literally uses an id obtained from the first, a waterfall is unavoidable — and that is normal, not an error.

Numerical example. getCourse() takes 500 ms and getReviews() takes 800 ms. With sequential awaits, total waiting time is about 1300 ms. With Promise.all, both requests start together, so the wait is close to 800 ms. The speedup does not come from a “faster Promise.all”; it comes from eliminating unnecessary idle time.

When Promise.all is not appropriate: if getReviews() requires course.id, which can only be obtained after getCourse(), the dependency is real. In this case, the waterfall reflects the data dependency and is correct.

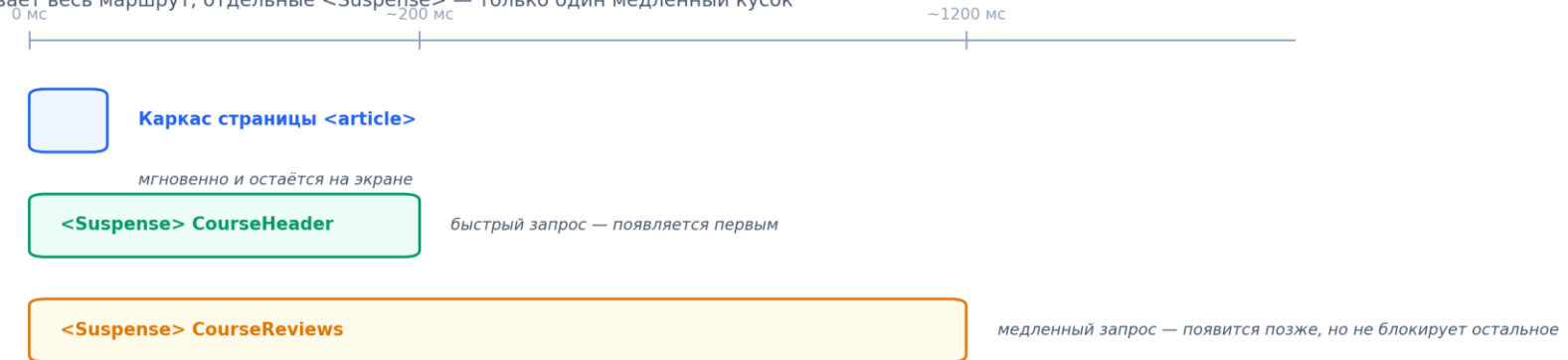
What if one of the requests is slow?

Parallel execution helps, but the page still will not appear until ALL awaits in the component have completed. If one request is consistently slow, a targeted <Suspense> boundary around that specific part allows the rest of the page to appear earlier:

Mental model for Suspense: it is not a “request accelerator.” The request remains just as slow. Suspense reduces perceived waiting time by allowing ready parts of the interface to appear earlier. The user can start reading the page while a secondary block is still loading.

Точечный Suspense: разные части страницы стримятся независимо

loading.tsx покрывает весь маршрут; отдельные `<Suspense>` — только один медленный кусок



Без вложенных `<Suspense>` вся страница ждала бы самый медленный запрос (1200 мс).
С ними пользователь видит заголовок курса почти сразу, а отзывы — когда будут готовы.

Diagram B. loading.tsx covers the entire route; individual `<Suspense>` boundaries stream independently

```

app/courses/[id]/page.tsx
import { Suspense } from "react";

export default function CoursePage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  return (
    <article>
      <Suspense fallback=<{<p>Loading course...</p></p>>
        <CourseHeader params={params} />
      </Suspense>
      <Suspense fallback=<{<p>Loading reviews...</p></p>>
        <CourseReviews params={params} />
      </Suspense>
    </article>
  );
}

```

CoursePage is no longer async here — it does not wait for anything itself; it only defines the boundaries. CourseHeader and CourseReviews are separate async components; each awaits params and performs its own data request. React streams each one exactly when it is ready.

Notice the architecture: the parent now handles composition and loading boundaries, while the child async components handle their own data. This separation is useful when different parts of the page load at different speeds and can be streamed independently.

4. When Server-Side Data Is Not Enough

A Server Component is excellent for the case where data is needed to display the page at all. But some scenarios require more: search-as-you-type without a page reload, an automatically refreshing feed, or an optimistic interface update before the server responds. This is where a client-side data layer is needed.

Selection criterion: if the data is needed to produce the initial HTML, start with a Server Component. If the data must react to user actions after the page has loaded — search-as-you-type, auto-refresh, optimistic UI — a client layer becomes necessary. These are not competing approaches; they serve different stages of the data lifecycle.

Server Component или SWR на клиенте?

Три вопроса, которые снимают большинство сомнений

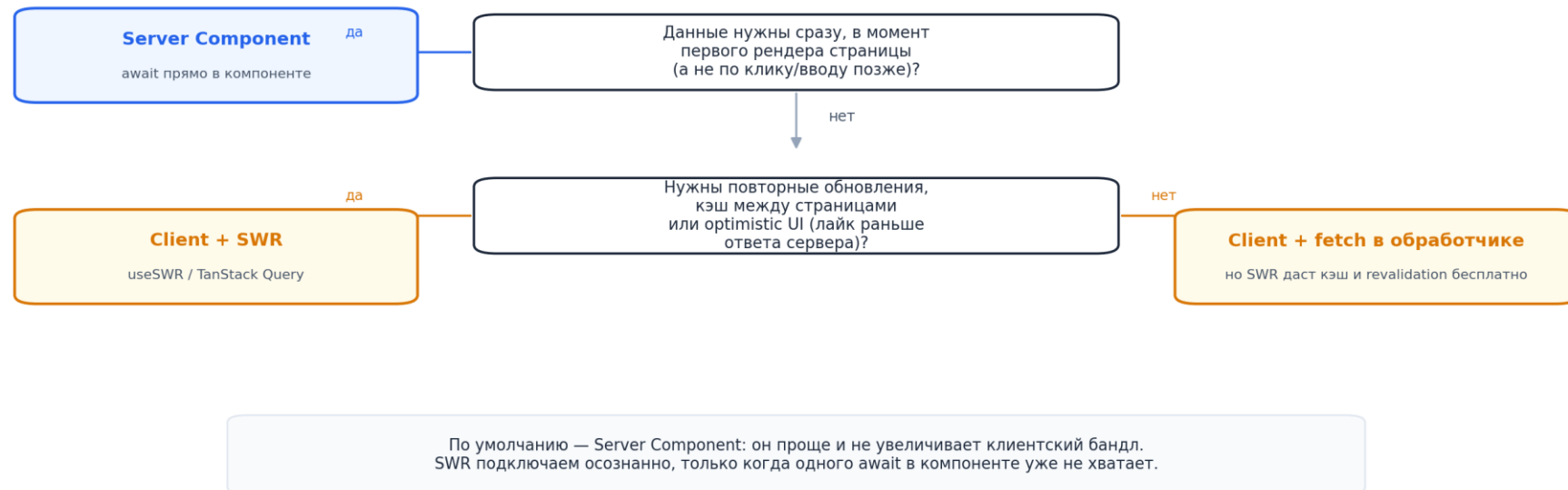


Diagram C. Server Component or client — two questions that resolve most doubts

Note: client-side data does not mean going back to `useEffect + fetch` from Section 2. It is a deliberate choice of a specialized tool — a library designed specifically for managing server state on the client.

5. SWR: Server State on the Client

SWR (from stale-while-revalidate) is a library from the Vercel team centered around a single hook: `useSWR`. The idea is that “server data displayed on the client” is a separate category of state with its own rules: it has a cache, it can become stale, and it may need to be requested again in the background. `useSWR` handles this instead of a manual `useState + useEffect` combination.

What stale-while-revalidate means in practice: the interface can immediately show a previously known value (stale), request a fresh version in parallel (revalidate), and then update the screen seamlessly. The user therefore does not have to see an empty screen on every transition.

It is important to distinguish UI state from server state. The query string in the search field is UI state because it belongs to the interface. Search results are server state because the server remains the source of truth. `useState` is well suited to the first; `SWR` is well suited to the second.

An alternative with the same purpose is TanStack Query (formerly React Query): it is more powerful for complex scenarios such as mutations, optimistic updates, and pagination, but also heavier. `SWR` is sufficient for this course: it is simpler, lighter, and comes from the same team as Next.js.

```

components/CourseSearch.tsx
"use client";
import { useState } from "react";
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((r) => r.json());

function CourseSearch() {
  const [query, setQuery] = useState("");

  const { data, isLoading } = useSWR(
    query ? `/api/courses/search?q=${query}` : null,
    fetcher
  );

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      {isLoading && <p>Searching...</p>}
      {data?.map((c: Course) => <p key={c.id}>{c.title}</p>)}
    </div>
  );
}

```

The key detail is the first argument to useSWR: while query is empty, null is passed, so SWR makes no request at all. As soon as the user starts typing, the key becomes a real URL, and SWR manages the request, cache, and repeated calls as query changes.

Step by step: query = "" → key is null → no request. The user types "re" → the key becomes /api/courses/search?q=re → SWR performs the request. The user continues with "react" → the key changes → SWR requests a different resource. In SWR, the key acts both as the data address and as the cache entry identifier.

6. Pattern: the Server Provides the Seed, the Client Takes Over (Advanced)

You may postpone this section: it combines Section 5 with what you already know about Server Components from Lecture 1, but using both together requires confidence in each part. If Section 5 is still new, it is reasonable to return here in a day or two.

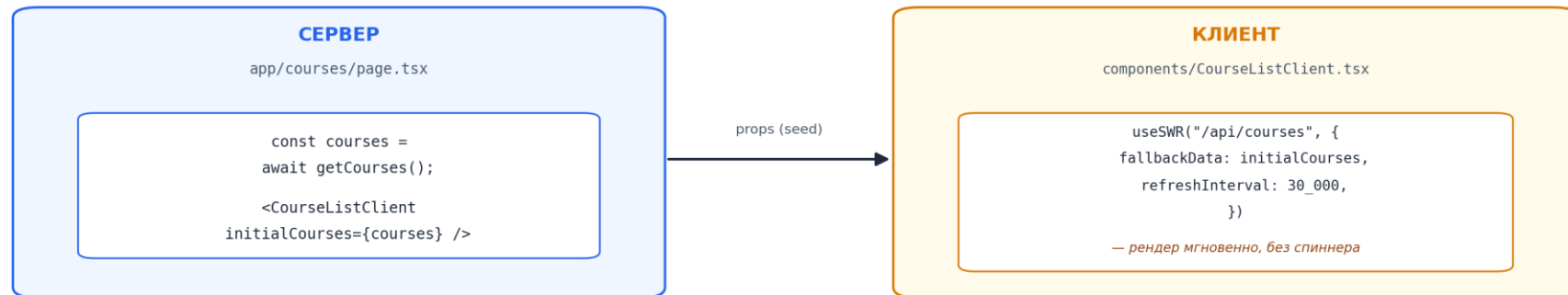
The most common beginner mistake with SWR is to use it everywhere, including cases where a Server Component could solve the task with a single await. A better hybrid is for the Server Component to load data once during the initial render and for the client to receive it as fallbackData — with no initial spinner — and then keep it fresh afterward.

Seed-pattern lifecycle: 1) the Server Component obtains courses; 2) HTML is immediately built with those data; 3) initialCourses are passed to the client component; 4) fallbackData lets SWR use them without an initial spinner; 5) after hydration, SWR can revalidate and update the data in the background.

Cost of the solution: as soon as a Client Component and SWR are introduced, part of the logic moves into client-side JavaScript. Therefore, the hybrid is appropriate where later refreshes provide real value, not “just in case.”

Паттерн: сервер отдаёт seed, SWR подхватывает на клиенте

Ни спиннера при первой загрузке, ни устаревших данных потом



Что происходит дальше:

t=0

Первый рендер — сразу показывает fallbackData, сети ещё не было

t=30с

SWR тихо перезапрашивает данные в фоне (refreshInterval)

при фокусе окна

SWR по умолчанию тоже ревалидирует — вкладку переключили и вернулись

Diagram D. The Server Component passes the seed; SWR takes over and refreshes in the background

```

app/courses/page.tsx (Server Component)
import { getCourses } from "@lib/courses";
import { CourseListClient } from "@components/CourseListClient";

export default async function CoursesPage() {
  const courses = await getCourses();

  return <CourseListClient initialCourses={courses} />;
}
  
```

```
components/CourseListClient.tsx (Client Component)
"use client";
import useSWR from "swr";
import type { Course } from "@/lib/courses";

const fetcher = (url: string) => fetch(url).then((r) => r.json());

function CourseListClient({
  initialCourses,
}: {
  initialCourses: Course[];
}) {
  const { data: courses } = useSWR<Course[]>("/api/courses", fetcher, {
    fallbackData: initialCourses,
    refreshInterval: 30_000,
  });

  return (
    <ul>
      {courses!.map((c) => <li key={c.id}>{c.title}</li>)}
    </ul>
  );
}
```

7. useReducer: When useState Starts to Grow

As long as state is a single value — a string, number, or boolean — `useState` works very well. But a course list can quickly accumulate several related fields: category, sorting, and perhaps a display mode. Keeping them in three separate `useState` calls works, but as related updates grow, the code becomes harder to read. `useReducer` collects all state transitions in one place:

When should you consider a reducer? Not simply when the number of `useState` calls grows, but when related transitions appear. For example, changing category should also reset page to 1; RESET should restore several fields at once; some actions should update several values consistently. A reducer collects these rules in one function.

```

components/CourseFilters.tsx
type FilterState = { category: string; sort: "popular" | "new" };
type FilterAction =
  | { type: "SET_CATEGORY"; category: string }
  | { type: "SET_SORT"; sort: "popular" | "new" }
  | { type: "RESET" };

const initialState: FilterState = { category: "all", sort: "popular" };

function filterReducer(state: FilterState, action: FilterAction): FilterState {
  switch (action.type) {
    case "SET_CATEGORY":
      return { ...state, category: action.category };
    case "SET_SORT":
      return { ...state, sort: action.sort };
    case "RESET":
      return initialState;
  }
}

```

Inside the component, instead of three `setState` functions, there is one `dispatch`: `dispatch({ type: "SET_SORT", sort: "new" })`. All update rules live in `filterReducer`, where they can be read and tested separately from the markup.

Useful model: `dispatch` describes the event (“what happened”), while the reducer describes the transition (“how the state changes”). The component does not need to know every detail of the update. This makes complex state easier to read, test, and extend.

8. `useContext`: Without Prop Drilling (Advanced)

You may postpone this section: `useContext` solves a specific problem — passing props through many levels. While your project has only two or three components, this problem probably has not appeared yet. Passing props through one or two levels is normal, not a sign of bad code. Return to this section when prop drilling actually becomes inconvenient.

The filters from Section 7 will likely be needed by several components at once — the card list, the result counter, and perhaps breadcrumbs. Passing filters and `dispatch` through props at every level works, but it is verbose. `useContext` provides direct access to the value without going through intermediate components:

When Context is not needed: if a value passes through only one or two levels, ordinary props are simpler and more transparent. Context becomes useful when the same data is needed by many components in different branches and intermediate components are forced only to forward props.

```
context/FilterContext.tsx
"use client";
import { createContext, useContext, useReducer } from "react";

const FilterContext = createContext<{
  state: FilterState;
  dispatch: React.Dispatch<FilterAction>;
} | null>(null);

export function FilterProvider({ children }: { children: React.ReactNode }) {
  const [state, dispatch] = useReducer(filterReducer, initialState);

  return (
    <FilterContext.Provider value={{ state, dispatch }}>
      {children}
    </FilterContext.Provider>
  );
}

export function useFilters() {
  const ctx = useContext(FilterContext);
  if (!ctx) throw new Error("useFilters used outside FilterProvider");
  return ctx;
}
```

Now any Client Component inside `<FilterProvider>` can obtain `{ state, dispatch }` in one line — `const { state, dispatch } = useFilters()` — regardless of how deeply nested it is.

Architectural rule: place the Provider as close as possible to the area that actually needs shared state. If filters belong only to the catalog, there is no reason to move `FilterProvider` to the root layout of the entire application.

Important: `createContext`, `useReducer`, and `useContext` require 'use client', like other client hooks. The Provider should wrap only the part of the tree that actually needs access to the filters, not the entire application.

Situation	First choice	Why
Data is needed for the initial HTML	Server Component	Less client-side JS; data arrives with the page
Several independent server requests	<code>Promise.all</code>	Eliminates unnecessary sequential waiting
One part of the page is noticeably slower	<code>Suspense</code>	Ready parts of the interface appear earlier
Data must refresh after page load	<code>SWR</code>	Cache, revalidation, and repeated client-side requests
One simple local value	<code>useState</code>	Minimal and straightforward tool
Related complex state	<code>useReducer</code>	Centralizes transition rules

Shared state deep in the tree	<code>useContext</code>	Avoids excessive prop drilling
Synchronization with browser/timer/subscription	<code>useEffect</code>	Connects React to an external system

9. Summary

- `useEffect` is for synchronization with external systems (subscriptions, timers, browser APIs), not for loading data when a Server Component can do the job.
- Independent awaits in a Server Component should be run in parallel with `Promise.all` rather than one after another.
- A targeted `<Suspense>` boundary around a slow section prevents it from blocking the rest of the page.
- Use SWR intentionally when repeated refreshes, cross-page caching, or optimistic UI are needed — not by default everywhere.
- `fallbackData` pattern: the server loads the data once, the client receives it without an initial spinner, and then keeps it fresh.
- `useReducer` brings order when several state fields change according to related rules; `useContext` shares that state without prop drilling.

10. Connection to Laboratory Work 2

In Laboratory Work 2, you will add client-side course catalog search to the project from Week 1. The required part is based on Section 5 of this lecture:

1. install `swr`;
2. move title search into a separate `CourseSearch` component using `useSWR`, as shown in Section 5;
3. verify that there is no `useEffect` with `fetch` inside anywhere in the project.

Additionally, if the main part is complete and time remains:

1. move the category filter and sorting to `useReducer` (Section 7) instead of separate `useState` calls;
2. implement the seed pattern from Section 6 for the main course list;
3. wrap the catalog page in `FilterProvider` and access filters through `useContext` (Section 8).

The completion criterion remains the same: `npm run build` must finish without TypeScript errors.

11. Self-Check Questions

1. Why is the `CourseStats` example using `useEffect` considered redundant in the App Router, and when is `useEffect` still justified?
2. How will response time change if `Promise.all` for two independent operations in a Server Component is replaced with sequential awaits?
3. Why did `CoursePage` in Section 3 stop being an async component after `Suspense` was introduced?

4. Give an example where client-side data with SWR is needed even though a Server Component could technically obtain the same data.
5. What happens on the first render of CourseListClient if fallbackData is removed from the useSWR call?
6. When is it worth replacing three useState calls with one useReducer?

12. Materials for Independent Study

- React — You Might Not Need an Effect — <https://react.dev/learn/you-might-not-need-an-effect>
- React — Synchronizing with Effects — <https://react.dev/learn/synchronizing-with-effects>
- React — useReducer (reference) — <https://react.dev/reference/react/useReducer>
- React — useContext (reference) — <https://react.dev/reference/react/useContext>
- Next.js — Loading UI and Streaming — <https://nextjs.org/docs/app/building-your-application/routing/loading-ui-and-streaming>
- SWR — official documentation — <https://swr.vercel.app>

